

TMF630 REST API Design Guidelines Part 4: Advanced guidelines for RESTful APIs: import and export tasks, entity versioning

TMF630 Part 4

Maturity Level: General Availability (GA)	Team Approved Date: September 4th, 2025
Release Status: Pre-production	Approval status: Team Approved
Version: 5.0.0	IPR Mode: RAND

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive
Suite 301 #1649
Rockaway, NJ 070866, USA
Tel No. +1 862 227 1648

TM Forum Web Page: www.tmforum.org

Table of Contents

Notice	2
Executive Summary	4
Conventions	5
Keywords to Indicate Requirement Levels	5
Formatting Conventions for Examples	5
1. Export Data Tasks	6
1.1. Export Job Resource	6
1.2. Creating Export Jobs	11
1.3. Query Export Jobs	14
1.4. Export Job State Change Notification	15
2. Import Data Tasks	17
2.1. Import Job Resource	17
2.2. Creating Import Jobs	21
2.3. Query Import Jobs	22
2.4. Import Job State Change Notification	23
3. Entity Versioning and Lifecycle Management	25
3.1. Design Requirements	26
3.2. Versioned Entity Model Pattern	27
3.3. Date Considerations	29
3.4. Version Property and Format	30
3.5. Versioned Entity Example	31
3.6. Versioned Entity Implementation	32
3.7. Operations on Versioned Entities	32
3.8. Events Related to Versioned Entities	49
3.9. When to create a new Version	54
3.10. Versions and History	55
3.11. Role Based Access Control	55
References	56
Administrative Appendix	58
Document History	58
Release History	58
Acknowledgments	58

Executive Summary

The “REST API Design Guidelines” document provides guidelines and design patterns used in designing TM Forum REST APIs. The document is organized in multiple parts as follows:

- **Part One:** Practical guidelines for RESTful APIs; naming, CRUD, filtering, tasks, history, notifications
- **Part Two:** Advanced guidelines for RESTful API polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- **Part Three:** Guidelines for extending TMF Open APIs with hypermedia support
- **Part Four:** Advanced guidelines for RESTful APIs: import and export tasks, entity versioning
- **Part Five:** JSON Patch extension to manage arrays
- **Part Six:** JSON Path extension
- **Part Seven:** JSON Schema patterns
- **Part Eight:** API Specialization

This part (Part 4) provides information for the development of TMForum APIs using REST.

It provides recommendations and guidelines for the implementation of patterns relating to common Export and Import Tasks and the Management of Entity Versions.

Conventions

Keywords to Indicate Requirement Levels

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [\[RFC8174\]](#).

Formatting Conventions for Examples

In most examples, attributes occur that normally would be long UUID or similar values; as simplification of the layout (avoidance of unnecessary line breaks), these values are shortened to a length of 4 characters only, followed by two dots:

`bd69..` instead of `bd694f23-627f-4553-a1df-7e79d62001ac`

For the same reason, in examples we abbreviate the leading part of URLs by a placeholder:

`{apiRoot}` instead of `https://mycsp.com/tmf-api`



As explained in [part 1 of the Guidelines](#), also relative paths to address APIs can be used in `href` attributes.

Chapter 1. Export Data Tasks

This section describes the common tasks used for exporting resource representations.

Common task resources is defined:

- `ExportJob` used to export resources

As export format a `json` document (content type `application/json`) that contains a collection of resources (a set of arrays, one array for each type of resources) **MUST** be supported. Other formats **MAY** be supported additionally.

By default, an export should be considered to be atomic, i.e. it either succeeds completely, or not at all, even if the export contains multiple resources and only for some of them an error occurred. If an error occurs during the export, the status of the export job **MUST** be set to "Failed" and the error log **MUST** contain information about the parts of the export data that failed.

1.1. Export Job Resource

An ***ExportJob*** resource represents a task (with a simplified status schema) used to export resource representations. It can be applied to any API either by the API author or by implementations when there is a need to do so.



Although being a task, `ExportJob` is not designed based on `TaskResource` via `<[OAS3] allOf` due its simpler lifecycle. See the section on general Task Resources in [Part 1 of the Guidelines](#).

To trigger the export of data maintained by an API that implements it, an `ExportJob` is created by the client (see [Creating Export Jobs](#)).

The *ExportJob* resource supports the following properties:

Attribute name	Description
path	JSON Pointer or JSON Path relative to the API root node that points to the resource acting as the source for streaming content to the export destination specified by the <i>ExportJob</i>. SHOULD be present in the request defaults to the API's root node. SHOULD be present in the response (if it was sent in the request) Examples (for the <i>ExportJob</i> on the Product Catalog API): JSON Pointer to export entities from the ProductCatalog with id 2027: <code>/productCatalog/2027</code> JSON Path to export entities from the "Virtual Storage Medium" ProductOffering: <code>\$.productOffering[?(@.name=="Virtual Storage Medium")]</code> NOTE: Using the JSON Pointer resp. JSON Path with reference to the API root guarantees that the path cannot point to a resource outside the scope of this API, e.g. to export some resource managed by a different API.

Attribute name	Description
query	Used to scope the exported data (identical to GET filter construct using the API's node defined by path as base) MAY be present in the request defaults to requesting all resources of the node defined by the path. SHOULD be present in the response (if it was sent in the request) Example: "query": "type=productOffering&version=2.0"
status	"Not Started", "Running", "Succeeded", "Failed" Generated by the system, not to be provided in the request As an export should be considered to be atomic, the status of the export job MUST "Failed" if any error occurred. MUST be present in the response
creationDate	Date at which the Job was created. Generated by the system, not to be provided in the request SHOULD be present in the response
completionDate	Date at which the Job was completed. Generated by the system, not to be provided in the request SHOULD be present in the response when the export is complete
contentType	The format of the exported data. MUST be present in the request if a different format than the default is used. defaults to "application/json"

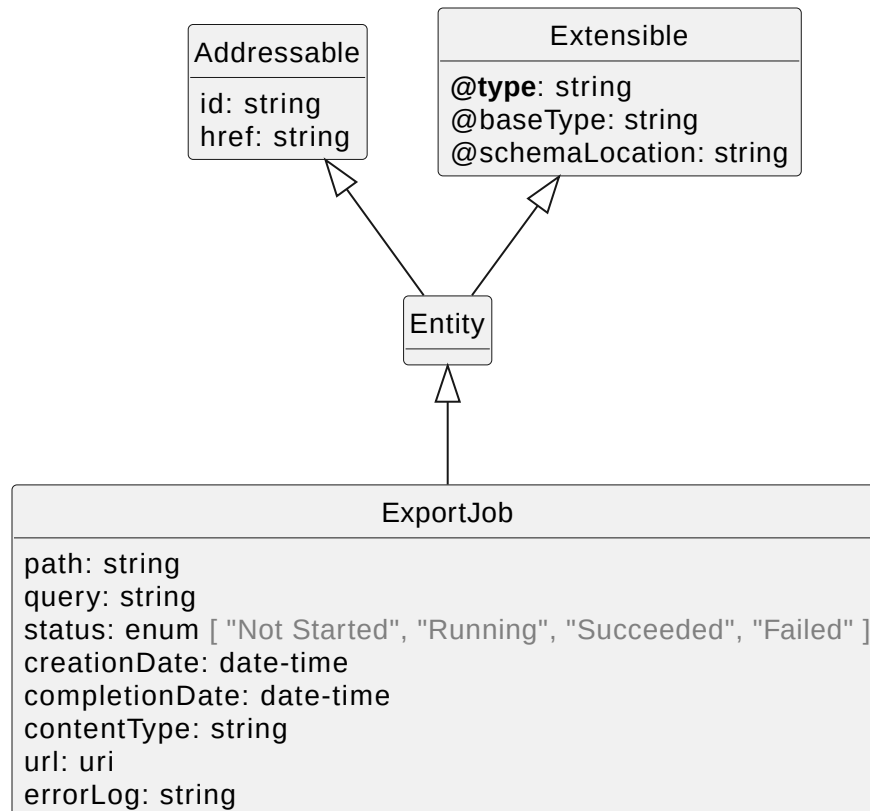
Attribute name	Description
url	URL pointing at the location for the exported data for file exports a URL of the form <code>file://host/path</code> where host is the fully qualified domain name of the system on which the path is accessible, and path is a hierarchical directory path of the form directory/directory/.../name. Other schemes can also be supported, e.g. "ftp" and "http", preferably with their TLS encrypted variants "sftp" resp "https" SHOULD be present in the request and in the response
errorLog	Reason for failure Generated by the system, not to be provided in the request SHOULD be present in the response <i>if an error occurred</i>. If only for part of the export an error occurred, the error log SHOULD contain information about the parts of the export data that failed.



The created export data SHOULD be protected against unauthorized access. For example, only the requestor or an otherwise entitled person should be allowed to download the exported file, HTTP resource or similar. The details of such security means is beyond this guideline.

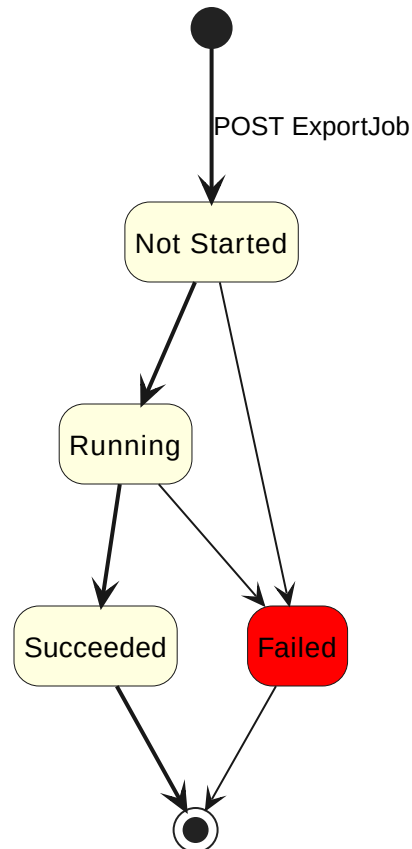
1.1.1. ExportJob schema

The following class diagram shows the structure of the ExportJob resource:



1.1.2. ExportJob lifecycle

The following state/transition diagram shows the lifecycle of the ExportJob resource:



1.2. Creating Export Jobs

ExportJob tasks are created as resources in the {apiRoot}/exportJob collection of the API.

1.2.1. Export Collection of Resources

It is possible to export collection of resources in an API, for example:

Export all the ProductOffering resources

REQUEST

```
POST {apiRoot}/productCatalogManagement/v5/exportJob
Content-Type: application/json
```

```
{
  "@type": "ExportJob",
  "path": "/productOffering",
  "url":
    "ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering.json"
```

```
}
```

RESPONSE

```
201 Created
Content-Type: application/json
Location: {apiRoot}/productCatalogManagement/v5/exportJob/45db..
```

```
{
  "id": "45db..",
  "href": "{apiRoot}/productCatalogManagement/v5/exportJob/45db..",
  "@type": "ExportJob",
  "status": "Running",
  "path": "/productOffering",
  "contentType": "application/json",
  "creationDate": "2024-03-20T16:49:40.154Z",
  "url":
    "ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering.json"
}
```



Property *errorLog* is only returned in the error case, and *completionDate* will only be returned when the export is complete.

1.2.2. Export a specific resource

Likewise, it also could be used to export a specific resource:

Export the "Catalog 42"



in this example, "Catalog 42" is assumed to be the name of the catalog, whereas the catalog's id is assumed to be **2027...**

REQUEST

```
POST {apiRoot}/productCatalogManagement/v5/exportJob
Content-Type: application/json
```

```
{
  "@type": "ExportJob",
```

```
"path": "/productCatalog/2027..",  
"url": "ftp://ftp.myCatalog.com/productCatalog/catalog42.json"  
}
```

RESPONSE

```
201 Created  
Content-Type: application/json  
Location: {apiRoot}/productCatalogManagement/v5/exportJob/86b5..
```

```
{  
  "id": "86b5..",  
  "href": "{apiRoot}/productCatalogManagement/v5/exportJob/86b5..",  
  "@type": "ExportJob",  
  "status": "Not Started",  
  "path": "/productCatalog/2027..",  
  "creationDate": "2024-03-20T16:49:40.154Z",  
  "contentType": "application/json",  
  "url": "ftp://ftp.myCatalog.com/productCatalog/catalog42.json"  
}
```

1.2.3. Export Resources using Attribute Filtering

It is also possible to use *attribute filtering* to export resources with specific characteristics, for example:

Export all the ProductOffering resources that have been updated after 01/01/2023

REQUEST

```
POST {apiRoot}/productCatalogManagement/v5/exportJob  
Content-Type: application/json
```

```
{  
  "@type": "ExportJob",  
  "path": "/productOffering",  
  "query": "lastUpdate.gt=2023-01-01",  
  "url":  
    "ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering20230101.j  
    son"
```

```
}
```

RESPONSE

```
201 Created
Content-Type: application/json
Location: {apiRoot}/productCatalogManagement/v5/exportJob/7876..
```

```
{
  "id": "7876..",
  "href": "{apiRoot}/productCatalogManagement/v5/exportJob/7876..",
  "@type": "ExportJob",
  "status": "Running",
  "path": "/productOffering",
  "query": "lastUpdate.gt=2023-01-01",
  "creationDate": "2024-03-20T16:49:40.154Z",
  "contentType": "application/json",
  "url":
"ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering20230101.j
son"
}
```

1.3. Query Export Jobs

ExportJob resources can be found under the API/exportJob collection and may be retrieved using the normal GET constructs.

For example:

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/exportJob/86b5..
Accept: application/json
```

RESPONSE

```
200 OK
Content-Type: application/json
```

```
{
  "id": "86b5..",
  "href": "{apiRoot}/productCatalogManagement/v5/exportJob/86b5..",
  "@type": "ExportJob",
  "status": "Succeeded",
  "path": "/productOffering",
  "query": "lastUpdate.gt=2023-01-01",
  "creationDate": "2024-03-20T16:49:40.154Z",
  "completionDate": "2024-03-20T16:52:32.930Z",
  "contentType": "application/json",
  "url":
"ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering20230101.j
son"
}
```

1.4. Export Job State Change Notification

This event provides notification that the state of an export task has changed.

For example:

REQUEST

```
POST {apiRoot}/client/listener
Accept: application/json
```

```
{
  "@type": "ExportJobStateChangeEvent",
  "eventId": "d21a..",
  "eventTime": "2024-03-20T16:52:34.029Z",
  "eventType": "ExportJobStateChangeEvent",
  "event": {
    "exportJob": {
      "id": "45db..",
      "href": "{apiRoot}/productCatalogManagement/v5/exportJob/45db..",
      "@type": "ExportJob",
      "status": "Succeeded",
      "path": "/productOffering",
      "query": "lastUpdate.gt=2023-01-01",
      "creationDate": "2024-03-20T16:49:40.154Z",
      "completionDate": "2024-03-20T16:52:32.930Z",
```

```
"contentType": "application/json",  
  "url":  
    "ftp://ftp.myCatalog.com/productCatalog/productOffering/productOffering.json"  
  }  
}
```



This assumes the consumer has subscribed successfully to the *ExportJobStateChangeEvent*, via

```
POST {apiRoot}/productCatalogManagement/v5/hub  
Accept: application/json
```

```
{  
  "callback": "{apiRoot}/client/listener",  
  "query": "eventType=ExportJobStateChangeEvent"  
}
```

Chapter 2. Import Data Tasks

This section describes the common tasks used for importing resource representations from a source, e.g. a file or HTTP resource. It can be applied to any API either by the API author or by implementations when there is a need to do so.



Although being a task, *ImportJob* is not designed based on *TaskResource* via <[\[OAS3\]](#) [allOf](#) due its simpler lifecycle. See the section on general Task Resources in [Part 1 of the Guidelines](#).

A common task resources is defined:

- *ImportJob* used to import resources

As import format a **json** document (content type **application/json**) that contains the collection of resources that is going to be imported (a set of arrays, one array for each type of resources) **MUST** be supported. Other formats **MAY** be supported additionally.

2.1. Import Job Resource

An *ImportJob* resource represents a task (with a simplified status schema) used to import resource representations.

To trigger the import of data maintained by an API that implements it, an *ImportJob* resource is created by the client (see [Creating Import Jobs](#)), providing the to be imported data.

If an import succeeds completely, the status of the import job **MUST** be set to "Succeeded". If any part of the import fails, the status **MUST** be "Failed", disregard of the extent of the failures. The error log **MUST** then contain information about the parts of the import data that failed.

The *ImportJob* resource supports the following properties:

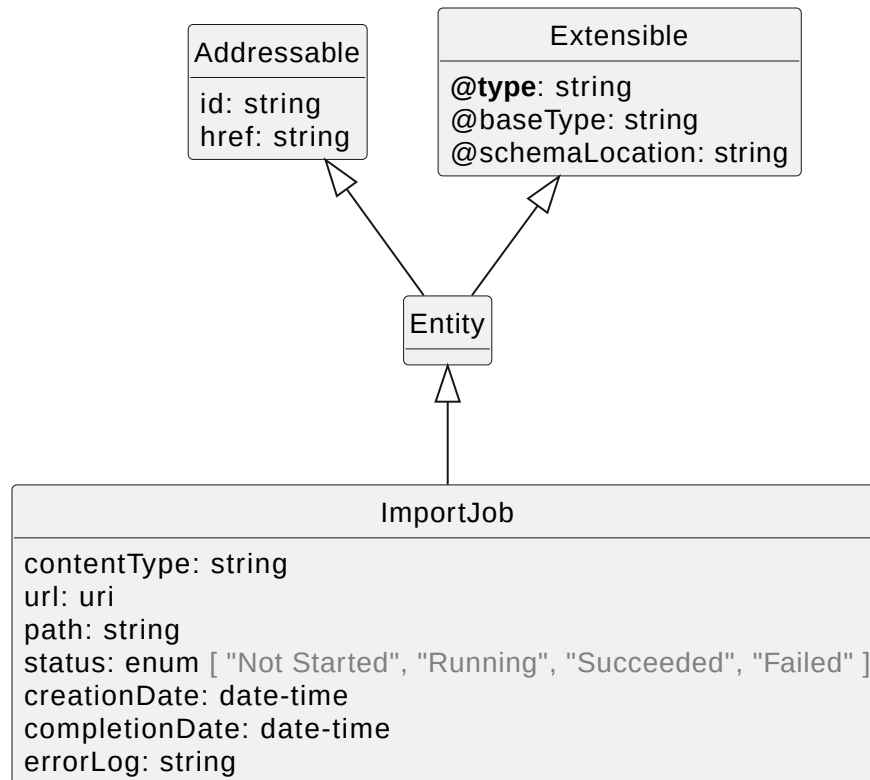
Attribute name	Description
contentType	The format of the imported data. MUST be present in the request if a different format than the default is used. defaults to application/json

url	URL pointing at the location for the data to be imported for file imports a URL of the form <code>file://host/path</code> where host is the fully qualified domain name of the system on which the path is accessible, and path is a hierarchical directory path of the form directory/directory/.../name. Other schemes can also be supported, e.g. "ftp" and "http", preferably with their TLS encrypted variants "sftp" resp. "https" SHOULD be present in the request and in the response
path	JSON Pointer or JSON Path relative to the API's root node that points to the root resource where the content of the import specified by the <i>ImportJob</i> must be applied MAY be present in the request SHOULD be present in the response defaults to the API's root node. Examples: (for the <i>ImportJob</i> on the Product Catalog API): JSON Pointer to import entities into the ProductCatalog with id=2027..: <code>/productCatalog/2027..</code> JSON Path to import entities into the "Virtual Storage Medium" ProductOffering: <code>\$.productOffering[?(@.name=="Virtual Storage Medium")]</code> Note: Using the JSON Pointer resp. JSON Path with reference to the API root guarantees that the path cannot point to a resource outside the scope of this API, e.g. to import some resource into a different API.

status	"Not Started", "Running", "Succeeded", "Failed" Generated by the system, not to be provided in the request MUST be present in the response If an import completely succeeds, the status MUST be "Succeeded". If any part of the import fails, the status MUST be "Failed", disregard of the extent of the failures.
creationDate	Date and time at which the Job was created. Generated by the system, not to be provided in the request SHOULD be present in the response
completionDate	Date and time at which the Job was completed. Generated by the system, not to be provided in the request SHOULD be present in the response when the Job is complete
errorLog	Reason for Failure if status is failed SHOULD be present in the response <i>if an error occurred</i> If only part of the import failed, the error log SHOULD contain information about the parts of the import data that failed.

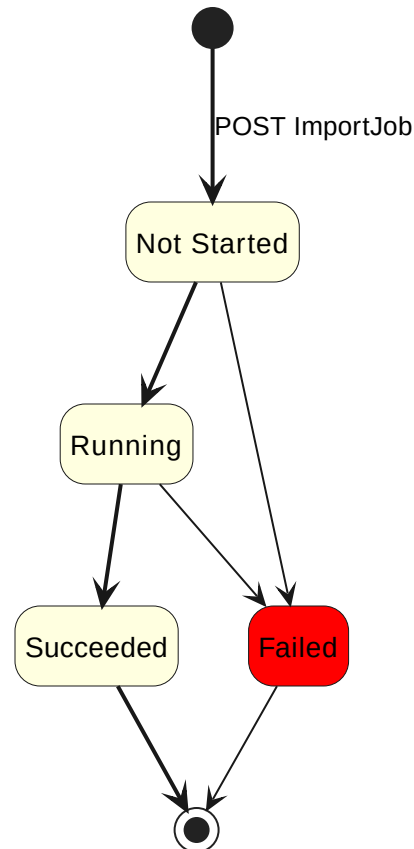
2.1.1. ImportJob schema

The following class diagram shows the structure of the ImportJob resource:



2.1.2. ImportJob lifecycle

The following state/transition diagram shows the lifecycle of the **ImportJob** resource:



2.2. Creating Import Jobs

ImportJob tasks are created as resources under the `{apiRoot}/importJob` collection.



The *ImportJob* **SHOULD** be used to import resources that were already exported, and it **SHOULD NOT** be used to create new resources using a File as input.

The main reason for this is that all the identification attributes, href and links of resources **SHALL NOT** be generated in the import job, this attributes **MUST** be sent in the import file.

For example, to apply the content of the import file to the catalog 10:



in this example, "Catalog 10" is assumed to be the name of the catalog, whereas the catalog's id is assumed to be **1305...**

REQUEST

```
POST {apiRoot}/productCatalogManagement/v5/importJob
```

```
Content-Type: application/json
```

```
{
  "@type": "ImportJob",
  "contentType": "application/json",
  "url": "ftp://ftp.myCatalog.com/productCatalog/catalog.json",
  "path": "/productCatalog/1305"
}
```

RESPONSE

```
201 Created
Content-Type: application/json
Location: {apiRoot}/productCatalogManagement/v5/importJob/6d26..
```

```
{
  "id": "6d26..",
  "href": "{apiRoot}/productCatalogManagement/v5/importJob/6d26..",
  "@type": "ImportJob",
  "contentType": "application/json",
  "url": "ftp://ftp.myCatalog.com/productCatalog/catalog.json",
  "path": "/productCatalog/1305..",
  "status": "Not Started",
  "creationDate": "2024-05-23T10:13:42.693Z"
}
```

2.3. Query Import Jobs

ImportJob resources can be found under the API/importJob collection and may be retrieved using the normal GET constructs.

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/importJob/6d26..
Accept: application/json
```

RESPONSE

```
200 OK
Content-Type: application/json
```

```
{
  "id": "6d26..",
  "href": "{apiRoot}/productCatalogManagement/v5/importJob/6d26..",
  "@type": "ImportJob",
  "contentType": "application/json",
  "url": "ftp://ftp.myCatalog.com/productCatalog/catalog.json",
  "path": "/productCatalog/1305..",
  "status": "Succeeded",
  "creationDate": "2024-05-23T10:13:42.693Z",
  "completionDate": "2024-05-23T14:53:03.501Z"
}
```

2.4. Import Job State Change Notification

This event provides a notification that the state of an import task has changed.

REQUEST

```
POST {apiRoot}/client/listener
Accept: application/json
```

```
{
  "@type": "ImportJobStateChangeEvent",
  "eventId": "d21a..",
  "eventTime": "2024-05-23T14:53:09.152Z",
  "eventType": "ImportJobStateChangeEvent",
  "event": {
    "importJob": {
      "id": "6d26..",
      "href": "{apiRoot}/productCatalogManagement/v5/importJob/6d26..",
      "@type": "ImportJob",
      "contentType": "application/json",
      "url": "ftp://ftp.myCatalog.com/productCatalog/catalog.json",
      "path": "/productCatalog/1305..",
      "status": "Succeeded",
      "creationDate": "2024-05-23T10:13:42.693Z",
      "completionDate": "2024-05-23T14:53:03.501Z"
    }
  }
}
```

```
}  
}  
}
```

This assumes the consumer has subscribed successfully to the `ImportJobStateChangeEvent`, via

```
POST {apiRoot}/productCatalogManagement/v5/hub  
Accept: application/json
```

```
{  
  "callback": "{apiRoot}/client/listener",  
  "query": "eventType=ImportJobStateChangeEvent"  
}
```

Chapter 3. Entity Versioning and Lifecycle Management

In many business contexts, there is a requirement to allow entities to be versioned. Depending on the context, there might also be a need to restrict access to older versions to particular security principals.

For example, in Product Lifecycle Management – PLM – the same Product Offering may exist in a Catalog in multiple versions, each version with a different version designation. It may be possible for an administrator to see all the existing versions or for a partner to see only a subset of all the existing versions.



Support of the versioning pattern described in this section is not mandatory. API Designers can require its support for specific APIs, but it is not a general requirement for all APIs. If not mandated on the API level, implementors can choose to support this pattern.

If an implementation supports entity versioning, it **MUST** implement the pattern as specified in this section.



Do not mix up the concept of **entity versioning** with the concept of **API versioning**.

The entity version designation is not dependent on the version number of the API that manages the entity. For example, in PLM the same Product Catalog API (running at a specific version number) may be used to retrieve entities with different PLM version designations.



Do not mix up the concept of **entity versioning** with the concept of **entity revisions** (History Pattern) on the other hand:

- For entities that support the History Pattern, entity revisions are entirely controlled by the provider of the API, and a revision is created automatically for every change made to the entity.
- For versioned entities, there is an explicit decision to create a new version:
 - Either by the consumer of the API, for example in product catalog management
 - Or by the provider of the API, for example in the product inventory where completion of an order could create a new version of the

Product entity

Note that the versions of a versioned resource can also be revisioned, i.e. have a history with automatically created revisions, as described in [Part 1](#) of the Guideline.



This pattern is not yet supported by the tooling for generating the API Specifications (OAS files), User Guides and other artifacts. Until the tooling is enhanced, API Designers and Implementors need to manually add the versioning pattern to the generated artifacts.

3.1. Design Requirements

An API may be designed to include the here defined versioning pattern. An API implementation may choose to support versions or not.

Consumers that are aware of versioning of an entity and consumers that are not aware of versioning of the same entity should be able to use the same API implementation in parallel.

To realize this freedom, the versioning pattern is built so that a consumer not being aware of the provider's support for versioning will not notice any change in the API behavior. The consumer will still be able to use an API implementation that supports versioning as if it were not supporting it.

Current Version

For each versioned entity, at any point in time, one and only one of its version is considered as the "current" version.

Retrieve versioned Entity

When a versioned entity is retrieved (either via its **id** or via a query), the current version of the entity is returned.

Retrieve specific Version of Entity

A specific version of a versioned entity can be explicitly retrieved.

Create Entity

When an entity is created, the provider will automatically create the first version of the entity, and set it as the current version.

Create new Version of Entity

Up to decision by the implementation, a new version of a versioned entity can either be created by the consumer of the API, or – e.g. subject to some automatically triggered

business policy – by the provider of the API.

The new version is created as clone of either a version selected by the consumer resp. provider, or if no base version is chosen, the current version. It does not automatically become the current version.

Make an existing Version current

Up to decision by the implementation, an existing version can be made "current" by either explicit consumer request, or by the provider, e.g. as a result of some automatic business policy triggered by e.g. a change to the entity's attributes. The so far current version is no longer current.

Delete Entity

When an entity is deleted, all versions of the entity are deleted.

Delete non-current Version

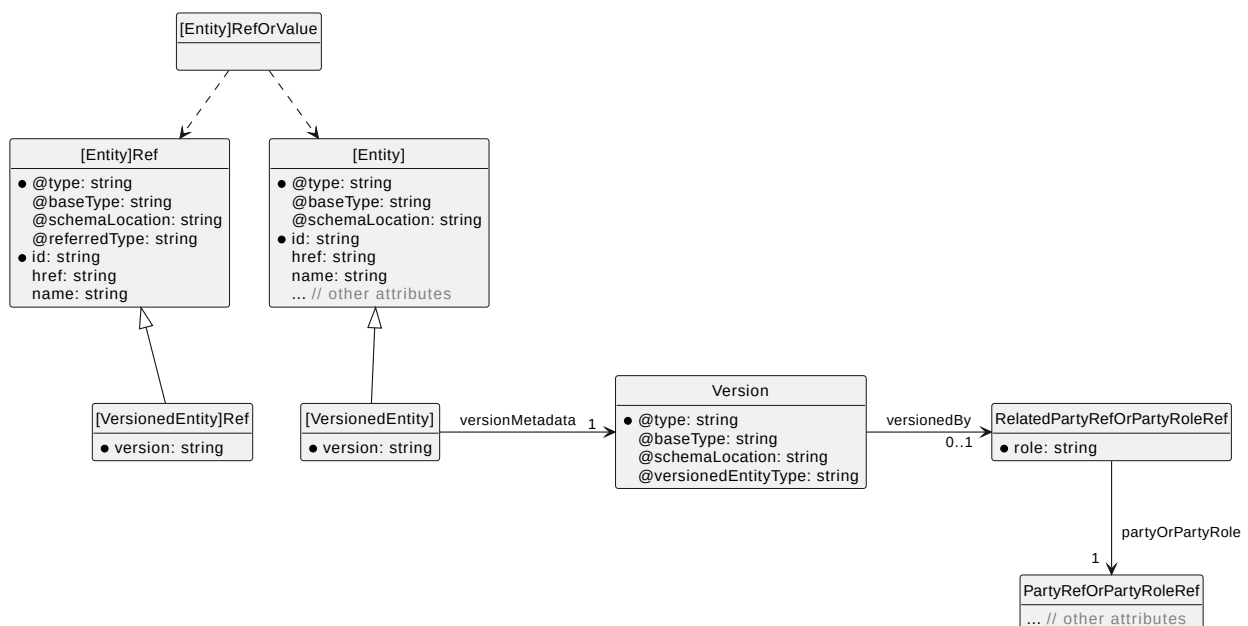
When a single version of an entity is deleted, and that version is not the current version, the version is deleted, and the current version remains unchanged.

Delete current Version

Deletion of the current version is not allowed, but rejected with **209 Conflict** error code.

3.2. Versioned Entity Model Pattern

In order to distinguish entities with a version designation, we propose the following conceptual model:



The salient points of this model:

3.2.1. Versioned Entity

This is a conceptual subclass of *Entity*, which in practice represents any top-level entity in the TM Forum Open API model that has versions. The key point is that a Versioned Entity has properties:

- *version*: a string with the semantic version of the entity, such as "1.1.0"
- *versionMetadata*: a **Version** sub-entity containing information about the version itself

Multiple instances of the entity can co-exist at the same time, with the same ID, but with different versions. In classic database terms, the primary key of the entity is the combination of *id* and *version*.



In V6 of the Open API, the plan is to remove the *version* property from versioned entities, since the same string is available in the *versionMetadata*. The *version* property should be considered as deprecated.



There is no expectation that the Open API model will be refactored to explicitly create this *VersionedEntity* subclass; the class is conceptual. In V6 this could be reconsidered for implementation.

3.2.2. Versioned Entity Reference

This is a conceptual subclass of *EntityRef*, which in practice represents any reference to a versioned entity. Here also, the entity reference for a versioned entity needs to include a version, since the entity reference is a foreign key to the referred entity.



There is no expectation that the Open API model will be refactored to explicitly create this *VersionedEntityRef* subclass; the class is conceptual. In v6 this could be reconsidered for implementation.

3.2.3. Version sub-entity

The **Version** sub-entity keeps metadata about the version (when was the version created, and by whom).

Version can also have a description that could be used (for instance) to explain why the version was created.

The value of the *version* property SHOULD be the same as the value of the *version* property in the versioned entity.



The related party reference is optional, since a version may have been created automatically by a software system.

In a future enhancement, we could consider adding a reference to *Resource* or even *SoftwareResource* to reflect this, and use *oneOf* party or resource in the model. This would be done in alignment with the Security API collection, where permissions and credentials can be applied to resources or resource roles as well as to parties or party roles.

isCurrent is a read-only calculated *Boolean* property that indicates if the version is considered as the current version.



At any point in time, there MUST be exactly one "current" Version instance of the same entity.

The implementation MUST ensure that the so far current version is no longer current when another version becomes current.

The semantics of being the current Version depends on the specific semantics of the versioned entity. For example:

- current version of **ProductOfferingPrice** would be the price version in effect on the date of the query
- current version of **ProductOffering** would be the offering version sellable on the date of the query

@versionedEntityType keeps the type of the versioned entity, for example *ProductOffering*. This can be useful when processing version-related events, as described below in the subsection on events.

@type, **@baseType**, **@schemaLocation** properties relate to the Version object

id, **href** properties relate to the versioned entity.

3.3. Date Considerations

The *Version* sub-entity has a **creationDate** property. This MUST be populated automatically when a version is created, and is not changeable afterwards.

Versioned entities MAY have *creationDate* and/or *lastUpdate* properties:

- The **creationDate** property reflects the date and time on which the first version of the entity was created, and MUST remain unchanged on all subsequent versions.

- The ***lastUpdate*** property will reflect the date and time on which the entity version was updated, and SHOULD have separate values for each version. Its initial value will correspond with the *Version* sub-entity's *creationDate*, but may change due to subsequent updates (PATCH) on a particular version of the entity.

Versioned entities MAY have a *validFor* property. This guideline does not define a causal connection between this property and the versioning. It is a design and implementation consideration how versions interact with validity periods.

For instance, it probably makes sense that each version of a *ProductOfferingPrice* has a different validity period, where the union of the periods of all the versions are contiguous (no gaps in coverage), while the intersection is empty (no overlaps). The "current" version might then be the one whose validity period contains the current point in time.

For other types of entity there may need to be different guidance.

3.4. Version Property and Format

The value of the ***version*** property represents the designation of the version. This value MUST be populated when a version is created:

- either by the consumer as input
- or set automatically by the provider

This is an implementation decision for each API. Thus, the property is optional for input but mandatory for output.

The value of version MUST be unique within the different versions of a given entity and must not be changed. It is a JSON string that SHOULD align with the [Semantic Versioning pattern](#) such as "4.2.1":

- Major version 4
- Minor version 2
- Patch version 1

It is up to the consumer or the provider to determine which changes in an entity warrant a new major version, or only minor version, or only patch.

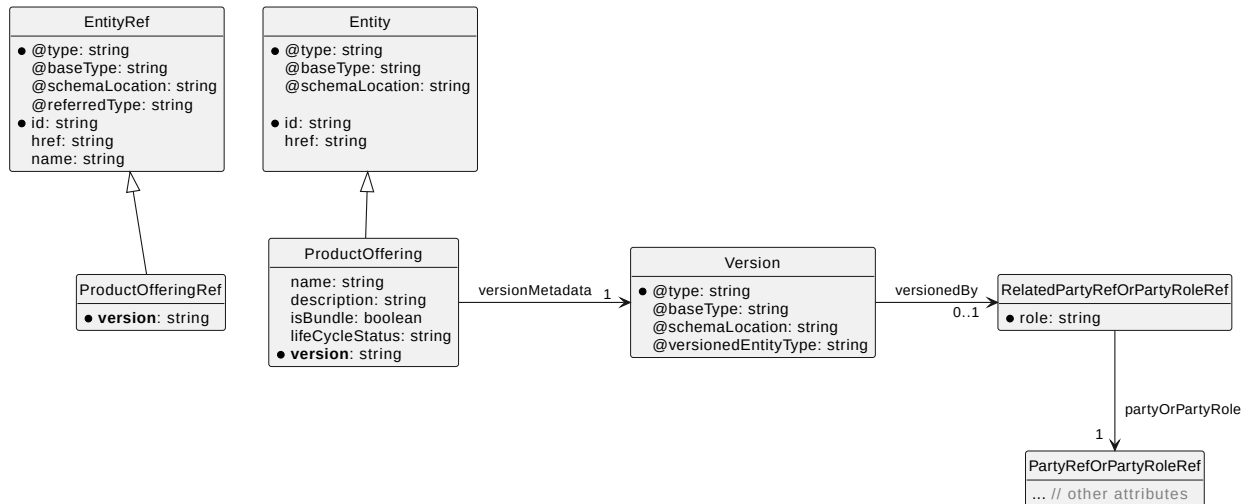
Taking a *ProductOffering* example:

- Fixing a spelling error in *name* would be a patch, so the version would become "4.2.2"
- Adding a new optional characteristic (e.g. *rank*) would be a minor change, so the version would become "4.3.0"
- Removing a bundled sub-offering would be a major change, so the version would

become "5.0.0"

3.5. Versioned Entity Example

The following diagram and code fragment show a model and example payload for *ProductOffering*, a versioned entity.



```

{
  "id": "4711..",
  "href":
    "{apiRoot}/productCatalogManagement/productOffering/4711..?version=1.0",
  "@type": "ProductOffering",
  "version": "1.0",
  "versionMetadata": {
    "id": "1123..",
    "href":
      "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
    "@type": "Version",
    "version": "1.0",
    "description": "Initial version",
    "isCurrent": false,
    "creationDate": "2013-04-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",

```

```
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    },
    "lastUpdate": "2013-04-19T16:42:23-04:00",
    "name": "Virtual Storage Medium",
    "description": "Virtual Storage Medium",
    "isBundle": "true",
    "lifecycleStatus": "Inactive"
  }
}
```

3.6. Versioned Entity Implementation

For Open APIs with versioned entities, each implementation can decide whether or not to implement the version pattern. Thus, the *version* and *versionMetadata* properties are optional, and will not be enforced by the CTK.

However, an implementation that chooses to implement the versioning pattern for an entity **MUST** do so as described in this section.

Specializations of entities should neither remove nor add the versioning pattern.

3.7. Operations on Versioned Entities

This section shows the expected endpoints that should be implemented for versioned entities.

In the endpoint examples we will assume that two versions of a "Virtual Storage" *Product Offering* exist in the Product Catalog; an "Inactive" and an "Active" version, respectively version "1.0" and version "2.0". The offering ID is "4711..".



For illustrative purposes most of the ProductOffering properties are omitted from the payload examples.

3.7.1. General Directive on Version in Query String

The **version** selector keyword can be used in the query string, where the value of the keyword is the semantic version:

?version=major.minor.patch

For example:

`?version=1.1.0`

Only the equals operator `=` is supported for the version selector. Other operators are not supported.

- If *version* selector is set to a specific version, the operation applies to that version of the entity.
- If *version* selector is not present, the operation applies to the current version of the entity.

This applies only when *version* is specified directly in the query string. If *version* is specified within a filter or [JSON Path](#) parameter, it will be treated like any other expression and will be applied to the *version* property of the entity.

If *version* is specified both as a selector and within a filter or [JSON Path](#) expression, the selector is applied first, specifying the scope of the filter or [JSON Path](#). For example:

- If *version* selector is set to "1.2.0" and then a filter expression contains `version==2.0.0` (contradicting the value on the selector), the result set will be empty.



Permissions are important for this case. It is likely that for non-current versions operations may be restricted to security principals with administrative permissions.

For example, authorized users of Product Catalog (e.g. administrators) typically have access to all the versions of catalog entities, while general users typically have access to only the current version of the entities, and support users in after-sales processes may need access to Product Offerings that are out of sale, but still used in older customer contracts.

3.7.2. Query all version metadata for an entity

The query looks like this:

```
GET {apiRoot}/{api}/{version}/{entity}/{id}/version
```

For example, the following request will return the metadata for all the versions of a particular product offering:

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/productOffering/4711../version
```

Accept: application/json

RESPONSE

200 OK
Content-Type: application/json

```
[
  {
    "id": "1123..",
    "href":
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
    "@type": "Version",
    "version": "1.0.0",
    "isCurrent": false,
    "creationDate": "2013-04-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    }
  },
  {
    "id": "4523..",
    "href":
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/4523..",
    "@type": "Version",
    "version": "2.0.0",
    "isCurrent": true,
    "creationDate": "2013-07-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
```

```
[
  {
    "id": "4434..",
    "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
    "name": "Joan Doe",
    "@type": "PartyRoleRef",
    "@referredType": "Employee"
  }
]
```

3.7.3. Querying versioned entities

When the GET list/search endpoint (on the entity collection) is invoked on a versioned entity, and the version selector is not specified as part of the filter expression, the result set will return only the current version of the entities that match the filter.

```
GET {apiRoot}/{api}/{entity}?{filter expression}
```



This is true even if the filter was by entity ID, so only a single entity satisfies the query; only the current version is returned.

For example, the following query will retrieve only current versions of Product Offerings that were last updated on 24 November 2023:

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/productOffering?lastUpdate.gte=2023-11-24T00:00:00&lastUpdate.lt=2023-11-25T00:00:00
Accept: application/json
```

RESPONSE

```
200 OK
Content-Type: application/json
```

```
[
  {
    "id": "4711..",
    "href":
```

```

"{apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=2.0",
  "@type": "ProductOffering",
  "version": "2.0",
  "versionMetadata": {
    "id": "4523..",
    "href":
"{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/4523..",
    "@type": "Version",
    "description": "Major update to the offering to adjust prices",
    "version": "2.0",
    "isCurrent": true,
    "creationDate": "2023-04-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    }
  },
  "lastUpdate": "2023-04-19T16:42:23-04:00",
  "name": "Virtual Storage Medium",
  "description": "Virtual Storage Medium",
  "isBundle": "true",
  "lifecycleStatus": "Active"
},
{
  "id": "3678..",
  "href":
"{apiRoot}/productCatalogManagement/v5/productOffering/3678..?version=2.9",
  "@type": "ProductOffering",
  "version": "2.9",
  "versionMetadata": {
    "id": "8836..",
    "href":
"{apiRoot}/productCatalogManagement/v5/productOffering/3678../version/8836..",
    "@type": "Version",
    "version": "2.9",
    "description": "Minor update to the offering to add explanatory materials",
    "isCurrent": true,

```

```

    "creationDate": "2023-04-10T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "9996..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/9996..",
        "name": "Ashok Patil",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    },
    "lastUpdate": "2023-04-10T16:42:23-04:00",
    "name": "Broadband for Students",
    "description": "Broadband for Students - cost-cutting offer for basic capabilities",
    "isBundle": "true",
    "lifecycleStatus": "Active"
  }
]

```

And the following query will return only the current version of the "Virtual Storage" *ProductOffering*:

REQUEST

```

GET {apiRoot}/productCatalogManagement/v5/productOffering?id=4711..
Accept: application/json

```

RESPONSE

```

200 OK
Content-Type: application/json

```

```

[
  {
    "id": "4711..",
    "href":
      "{apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=2.0",

```

```

    "@type": "ProductOffering",
    "version": "2.0",
    "versionMetadata": {
      "id": "3678..",
      "href":
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/3678..",
      "@type": "Version",
      "version": "2.0",
      "description": "Major update to the offering to adjust prices",
      "isCurrent": true,
      "creationDate": "2023-04-19T16:42:23-04:00",
      "@versionedEntityType": "ProductOffering",
      "versionedBy": {
        "role": "catalogAdmin",
        "@type": "RelatedPartyRefOrPartyRoleRef",
        "partyOrPartyRole": {
          "id": "4434..",
          "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
          "name": "Joan Doe",
          "@type": "PartyRoleRef",
          "@referredType": "Employee"
        }
      }
    },
    "lastUpdate": "2023-04-19T16:42:23-04:00",
    "name": "Virtual Storage Medium",
    "description": "Virtual Storage Medium",
    "isBundle": "true",
    "lifecycleStatus": "Active"
  }
]

```

If the *version* selector is specified, then the filtered results will include entity versions that match the selector. In practice, applying a search filter that includes a *version* selector is unlikely to be useful, unless multiple entities of the same type are versioned in lock-step to the same semantic version.

3.7.4. Retrieve a specific version of a versioned entity

A suitably authorized user can retrieve a specific version by specifying the *version* selector on the GET retrieve query string. The query looks like this:

```
GET {apiRoot}/{api}/{entity}/{id}?version={version}
```

For example, to retrieve version 1.0 of the "Virtual Storage" *ProductOffering*:

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=1.0.0
Accept: application/json
```

RESPONSE

```
200 OK
Content-Type: application/json
```

```
{
  "id": "4711..",
  "href":
  "{apiRoot}/productCatalogManagement/productOffering/4711..?version=1.0.0",
  "@type": "ProductOffering",
  "version": "1.0",
  "versionMetadata": {
    "id": "1123..",
    "href":
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
    "@type": "Version",
    "version": "1.0",
    "description": "Initial version",
    "isCurrent": false,
    "creationDate": "2013-04-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    }
  },
  "lastUpdate": "2013-04-19T16:42:23-04:00",
  "name": "Virtual Storage Medium",
```

```
"description": "Virtual Storage Medium",  
"isBundle": "true",  
"lifecycleStatus": "Inactive"  
}
```

3.7.5. Retrieve current version of an entity

When the regular GET retrieve endpoint is invoked on a versioned entity, the current version is returned. As described above, each API implementation needs to determine and document which version is considered current.

```
GET {apiRoot}/{api}/{entity}/{id}
```

For example, retrieving the current version of the "Virtual Storage" *ProductOffering*:

REQUEST

```
GET {apiRoot}/productCatalogManagement/v5/productOffering/4711..  
Accept: application/json
```

RESPONSE

```
200 OK  
Content-Type: application/json
```

```
{  
  "id": "4711..",  
  "href":  
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=2.0",  
  "@type": "ProductOffering",  
  "version": "2.0",  
  "versionMetadata": {  
    "id": "3678..",  
    "href":  
      "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/3678..",  
    "@type": "Version",  
    "version": "2.0",  
    "description": "Major update to the offering to adjust prices",  
    "isCurrent": true,  
    "creationDate": "2023-04-19T16:42:23-04:00",  
  },  
}
```

```

    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    },
    "lastUpdate": "2023-04-19T16:42:23-04:00",
    "name": "Virtual Storage Medium",
    "description": "Virtual Storage Medium",
    "isBundle": "true",
    "lifecycleStatus": "Active"
  }
}

```

3.7.6. Create first version of entity

POST on the entity collection is used to create the first version of a versioned entity. The request body SHOULD contain the Version sub-entity populated with the POSTable metadata of the initial version:

- the *version* string
- the *@type* and *@versionedEntityType*
- optionally the *description*
- optionally the party or party role that initiated the creation

The response body SHOULD contain the *Version* sub-entity populated with all the properties, including setting *isCurrent* to true (since there is only one version of the entity after initial creation).

REQUEST

```

POST {apiRoot}/productCatalogManagement/v5/productOffering
Accept: application/json

```

```
{
```

```

"@type": "ProductOffering",
"version": "1.0.0",
"versionMetadata": {
  "@type": "Version",
  "version": "1.0.0",
  "description": "Initial creation",
  "@versionedEntityType": "ProductOffering",
  "versionedBy": {
    "role": "catalogAdmin",
    "@type": "RelatedPartyRefOrPartyRoleRef",
    "partyOrPartyRole": {
      "id": "4434..",
      "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
      "name": "Joan Doe",
      "@type": "PartyRoleRef",
      "@referredType": "Employee"
    }
  }
},
"name": "Virtual Storage Medium",
"description": "Virtual Storage Medium",
"isBundle": "true",
"lifecycleStatus": "Active"
}

```

RESPONSE

201 Created
Content-Type: application/json

```

{
  "id": "4711..",
  "href":
    "{apiRoot}/productCatalogManagement/productOffering/4711..?version=1.0.0",
  "@type": "ProductOffering",
  "version": "1.0.0",
  "versionMetadata": {
    "id": "1123..",
    "href":
      "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
    "@type": "Version",
    "version": "1.0",
  }
}

```

```
"description": "Initial creation",
"isCurrent": true,
"creationDate": "2013-04-19T16:42:23-04:00",
"@versionedEntityType": "ProductOffering",
"versionedBy": {
  "role": "catalogAdmin",
  "@type": "RelatedPartyRefOrPartyRoleRef",
  "partyOrPartyRole": {
    "id": "4434..",
    "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
    "name": "Joan Doe",
    "@type": "PartyRoleRef",
    "@referredType": "Employee"
  }
},
"lastUpdate": "2013-04-19T16:42:23-04:00",
"name": "Virtual Storage Medium",
"description": "Virtual Storage Medium",
"isBundle": "true",
"lifecycleStatus": "Active"
}
```

3.7.7. Create new version of existing entity

POST is used to create a new version of an existing entity, on the *version* endpoint. The request body contains the *Version* sub-entity populated with the metadata of the new version.

If the *version* attribute is populated by the consumer, its value must differ from any existing version.

Creating this new version implicitly creates a clone of the versioned entity, with the same values for any functional properties. However, for special properties such as *validFor* and *status*, each entity will need specific rules for setting the values in the cloned entity. This might include automatic setting of values for such properties, or relaxing the PATCHable setting for those properties so that the consumer can provide the values.

The entity to be cloned can be the current version or any other existing version. To clone the current version, the POST endpoint uses the reserved keyword */version*:

```
POST {apiRoot}/{api}/{entity}/{id}/version
Content-Type: application/json
```

```
{
  "@type": "Version",
  "description": "<description>",
  "version": "<newVersion>",
  "versionedBy": {<RelatedParty>}
}
```

To clone a specific version, POST endpoint uses the selector *version={version}*:

```
POST {apiRoot}/{api}/{entity}/{id}?version={version}
Content-Type: application/json
```

```
{
  "@type": "Version",
  "description": "<description>",
  "version": "<newVersion>",
  "versionedBy": {<RelatedParty>}
}
```

For example, creating version 3.0.0 of the "Virtual Storage" *ProductOffering* from the current version:

REQUEST

```
POST {apiRoot}/productCatalogManagement/v5/productOffering/4711../version
Content-Type: application/json
```

```
{
  "@type": "Version",
  "version": "3.0.0",
  "description": "Major enhancement to Virtual Storage, allowing RAID and secure backups",
  "versionedBy": {<RelatedParty>}
}
```

RESPONSE

```
201 Created
```

Content-Type: application/json

```
{
  "id": "4571..",
  "href":
  "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/4571..",
  "@type": "Version",
  "creationDate": "2023-12-15T16:42:23-04:00",
  "version": "3.0.0",
  "description": "Major enhancement to Virtual Storage, allowing RAID and secure
backups",
  "isCurrent": false
}
```

And creating version 2.0.1 of the "Virtual Storage" *ProductOffering* from version 2.0.0:

REQUEST

POST {apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=2.0.0
Content-Type: application/json

```
{
  "@type": "Version",
  "version": "2.0.1",
  "description": "Patch to Virtual Storage, to allow correction of description
text",
  "versionedBy": {<RelatedParty>}
}
```

RESPONSE

201 Created
Content-Type: application/json

```
{
  "id": "2352..",
  "href":
  "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/2352..",
  "@type": "Version",
  ...
}
```

```
"creationDate": "2023-12-15T16:42:23-04:00",
"version": "2.0.1",
"description": "Patch to Virtual Storage, to allow correction of description
text",
"isCurrent": false
}
```



The new version is not automatically set to be the "current" version, this has to be made explicit by a successive PATCH. Depending on the API design, the PATCH may explicitly modify the *isCurrent* property, or it may do so implicitly by changing any other attribute on which "being current" depends according to business rules.



When one version becomes the current one, the version that so far was current **MUST NOT** stay current.

3.7.8. Modify an existing version of an entity

PATCH on the regular endpoint of the entity will act on the current version of the entity. For example, `PATCH {apiRoot}/.../productOffering/4711..` will update the Virtual Storage ProductOffering at Version 2.0.0 (assuming that this is the current version).

To update a specific version of an entity, a PATCH is executed on the entity with the *version* selector in the query string:

```
PATCH {apiRoot}/{api}/{entity}/{id}?version={version}
Content-Type: application/merge-patch+json
```

```
{
  "someProperty": "newValue",
  ...
}
```

The *version* property and the *Version* sub-entity itself are not patchable.

For example, to change the "Virtual Storage" *ProductOffering* versioned at 1.0.0 we would use:

REQUEST

```
PATCH {apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=1.0.0
Content-Type: application/json-patch+json
```

```
{
  "op": "replace",
  "path": "/description",
  "value": "Virtual Storage Medium Legacy"
}
```

RESPONSE

```
200 OK
Content-Type: application/json
```

```
{
  "id": "4711..",
  "href":
  "{apiRoot}/productCatalogManagement/productOffering/4711..?version=1.0.0",
  "@type": "ProductOffering",
  "version": "1.0.0",
  "versionMetadata": {
    "id": "1123..",
    "href":
    "{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
    "@type": "Version",
    "version": "1.0.0",
    "description": "Initial creation",
    "isCurrent": false,
    "creationDate": "2013-04-19T16:42:23-04:00",
    "@versionedEntityType": "ProductOffering",
    "versionedBy": {
      "role": "catalogAdmin",
      "@type": "RelatedPartyRefOrPartyRoleRef",
      "partyOrPartyRole": {
        "id": "4434..",
        "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
        "name": "Joan Doe",
        "@type": "PartyRoleRef",
        "@referredType": "Employee"
      }
    }
  }
}
```

```
}  
},  
"lastUpdate": "2013-04-19T16:42:23-04:00",  
"name": "Virtual Storage Medium",  
"description": "Virtual Storage Medium Legacy",  
"isBundle": "true",  
"lifecycleStatus": "Inactive"  
}
```

3.7.9. Deleting Versions

There is no DELETE on the version endpoint. The main entity DELETE endpoint can be used to remove a specific version – provided that this is not the current version – using the *version* selector on the query string.

```
DELETE {apiRoot}/{api}/{apiVersion}/{entity}/{id}?version={version}
```

For example, to delete version 1.0.0 of the "Virtual Storage" *ProductOffering*:

Request:

```
DELETE  
{apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=1.0.0
```

Response:

```
204 No Content
```

When the targeted version is the current version of the entity, the request **MUST** be rejected with HTTP status **409 Conflict**. The response body **SHOULD** contain a message indicating that the current version cannot be deleted.

For example, to delete current version 2.0.0 of the "Virtual Storage" *ProductOffering*:

Request:

```
DELETE  
{apiRoot}/productCatalogManagement/v5/productOffering/4711..?version=2.0.0
```

Response:

409 Conflict

```
{
  "@type": "Error",
  "code": "409",
  "reason": "deletion of current version is not allowed"
}
```

When the DELETE endpoint is invoked without the *version* selector, the entity is deleted, including all versions, in line with the requirement to provide the identical behavior as if the entity was not versioned or the API implementation did not support the versioning pattern:

Request:

DELETE {apiRoot}/productCatalogManagement/v5/productOffering/4711..

Response:

204 No Content

3.8. Events Related to Versioned Entities

The standard events that are raised for a versioned entity **MUST** contain information about the version to which the event applies, specifically the *version* property.

For example, the *StateChangeEvent* for a *ProductOffering* might appear as follows:

```
{
  "correlationId": "d991dd5c-cb42",
  "description": "ProductOfferingStateChangeEvent illustration, including version information",
  "domain": "Commercial",
  "eventId": "4768-a3c5-c47b06010b06",
  "eventTime": "2022-08-25T12:17:59.625Z",
  "eventType": "ProductOfferingStateChangeEvent",
  "priority": "4",
  "timeOccured": "2022-08-25T12:17:58.436Z",
}
```

```

"title": "ProductOfferingStateChangeEvent",
"event": {
  "productOffering": {
    "id": "4711..",
    "href":
"{apiRoot}/productCatalogManagement/productOffering/4711..?version=1.0",
    "@type": "ProductOffering",
    "version": "1.0",
  }
},
"reportingSystem": {
  "id": "868",
  "name": "APP-394",
  "@type": "ReportingResource",
  "@referredType": "LogicalResource"
},
"source": {
  "id": "372",
  "name": "APP-92",
  "@type": "ReportingResource",
  "@referredType": "LogicalResource"
},
"@baseType": "Event",
"@type": "ProductOfferingStateChangeEvent"
}

```

In addition to the standard entity events, versioned entities **SHOULD** raise events for version creation and version becoming current.

3.8.1. Entity Version Creation

The **[Entity]VersionCreateEvent** event should be raised whenever a new version of an entity is created, for example by POST on the version collection. The payload of the event is the new version of the entity, which includes the version metadata.



The event name is constructed from the entity name and the event type, in the same way as for all the events in the Open API model. So for *ProductOffering* the event will be *ProductOfferingVersionCreateEvent*.

For example, the event for a new version 2.0 of the "Virtual Storage" *ProductOffering* might appear as follows (for clarity, most of the *ProductOffering* payload is not shown):

```
{
```

```

"correlationId": "be58c421-12cf",
"description": "ProductOfferingVersionCreateEvent illustration",
"domain": "Commercial",
"eventId": "4a0a-9a52-86f728494b4e",
"eventTime": "2023-04-19T16:42:23-04:00",
"eventType": "ProductOfferingVersionCreateEvent",
"priority": "3",
"timeOccurred": "2023-04-19T16:42:23-04:00",
"title": "ProductOfferingVersionCreateEvent",
"event": {
  "productOffering": {
    "id": "4711..",
    "href": "{apiRoot}/productCatalogManagement/v5/productOffering/4711..",
    "@type": "ProductOffering",
    .... the rest of the ProductOffering payload ....
    "version": "2.0",
    "versionMetadata": {
      "id": "1123..",
      "href":
"{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
      "version": "2.0",
      "description": "Major update to the offering to set new prices",
      "isCurrent": true,
      "creationDate": "2023-04-19T16:42:23-04:00",
      "@versionedEntityType": "ProductOffering",
      "versionedBy": {
        "role": "catalogAdmin",
        "@type": "RelatedPartyRefOrPartyRoleRef",
        "partyOrPartyRole": {
          "id": "4434..",
          "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
          "name": "Joan Doe",
          "@type": "PartyRoleRef",
          "@referredType": "Employee"
        }
      }
    }
  }
},
"reportingSystem": {
  "id": "759",
  "name": "APP-745",
  "@type": "ReportingResource",
  "@referredType": "LogicalResource"
},

```

```

"source": {
  "id": "705",
  "name": "APP-317",
  "@type": "ReportingResource",
  "@referredType": "LogicalResource"
},
"@baseType": "Event",
"@type": "ProductOfferingVersionCreateEvent"
}

```

3.8.2. Entity Version Current

The **[Entity]VersionCurrentEvent** should be raised whenever a version of an entity becomes the current version, for example:

- by DELETE of the current version, so that according to the business rules a different version becomes current
- by PATCH of a versioned entity, so that according to the business rules a different version becomes current
- by passing of time, so that according to the business rules a different version becomes current

The payload of the event is the versioned entity minimum fields including version metadata, for that version of the entity that became the current one.



The event name is constructed from the entity name and the event type, in the same way as for all the events in the Open API model. So for *ProductOffering* the event will be *ProductOfferingVersionCurrentEvent*.

For example, the event for version 2.0 of the Virtual Storage product offering becoming current might appear as follows:

```

{
  "correlationId": "be58c421-12cf",
  "description": "ProductOfferingVersionCurrentEvent illustration",
  "domain": "Commercial",
  "eventId": "4a0a-9a52-86f728494b4e",
  "eventTime": "2023-04-19T16:42:23-04:00",
  "eventType": "ProductOfferingVersionCurrentEvent",
  "priority": "3",
  "timeOccured": "2023-04-19T16:42:23-04:00",
  "title": "ProductOfferingVersionCurrentEvent",
}

```

```

"event": {
  "productOffering": {
    "id": "4711..",
    "href": "{apiRoot}/productCatalogManagement/v5/productOffering/4711..",
    "@type": "ProductOffering",
    "version": "2.0",
    "versionMetadata": {
      "id": "1123..",
      "href":
"{apiRoot}/productCatalogManagement/v5/productOffering/4711../version/1123..",
      "@type": "Version",
      "description": "Major update to the offering to set new prices",
      "version": "2.0",
      "isCurrent": true,
      "creationDate": "2023-04-19T16:42:23-04:00",
      "@versionedEntityType": "ProductOffering",
      "versionedBy": {
        "role": "catalogAdmin",
        "@type": "RelatedPartyRefOrPartyRoleRef",
        "partyOrPartyRole": {
          "id": "4434..",
          "href": "{apiRoot}/partyRoleManagement/v5/partyRole/4434..",
          "name": "Joan Doe",
          "@type": "PartyRoleRef",
          "@referredType": "Employee"
        }
      }
    }
  },
  "reportingSystem": {
    "id": "759",
    "name": "APP-745",
    "@type": "ReportingResource",
    "@referredType": "LogicalResource"
  },
  "source": {
    "id": "705",
    "name": "APP-317",
    "@type": "ReportingResource",
    "@referredType": "LogicalResource"
  },
  "@baseType": "Event",
  "@type": "ProductOfferingVersionCurrentEvent"

```

```
}
```

3.9. When to create a new Version

The decision when to create a new version of an entity is not part of the API design, but rather depends on the business rules and processes of the organization that implements the API. The API design provides the mechanisms to create, retrieve, and manage versions, but it does not dictate when or why a new version should be created.

Business rules may relate that decision to e.g. the business semantic of the current status (e.g. to change a ProductOffering already in sale might require a new Version, and that will need to become the current one).

The following gives an example how actions on a ProductOffering might or might not trigger creation of a new Version; note that this is only one possible set of business rules that may be appropriate for one service provider, and other service providers might apply other rules:

- **change the status from "inTest" to "Launched"**
no new Version
- **change of the "description" attribute**
no new Version if not yet in sale;
new Version when in sale or already out of sale, but customers still have Products based on that ProductOffering; the version on which the existing Product instances are based must not be changed.
- **add new ProductOfferingPrice or change ProductOfferingPrice value**
analogous to the previous case
- **add a bundled offering inside the ProductOffering**
probably no new Version, as existing Product instances are not impacted; when customer requests change of his Product, he gets more options than before, so not a "breaking change" for the customer
- **change of the target ProductSpecification of the ProductSpecificationRef**
depends on ProductOffering status and whether the new ProductSpecification has more, less, or same constituents; same, better, ... service or resource, CharacteristicSpececation(Values): existing Product instances sold to customers must match the new ProductSpecification.

All in all: Whether a change of a versioned entity requires a new Version or not cannot be defined as part of the API's specification; the business rules controlling that decision might be implemented in the service (e.g. ODA component), but might also involve dynamic

policies or even manual decision by e.g. product manager.

3.10. Versions and History

The version and history patterns are completely orthogonal. For a versioned entity that also maintains history, the history entries will include the version metadata as part of each snapshot. Specifically:

- Creation of a new version will result in a history entry
- The history retrieval GET endpoint will behave as described above with respect to the *version* selector:
 - If *version* selector is not specified on the GET endpoint, retrieved history items will be for the current version
 - If *version* selector is specified, retrieved history items will be for that version

3.11. Role Based Access Control

Access control is now handled by the Access Control API suite, including the Roles and Permissions API TMF672 and the Digital Identity API TMF720.

References

- [RFC6901]
JavaScript Object Notation (JSON) Pointer
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc6901.html>
- [RFC8174]
Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc8174.html>
- [RFC9535]
JSONPath – Query Expressions for JSON
IETF (Internet Engineering Task Force)
<https://www.ietf.org/rfc/rfc9535.html>
- [SemVer]
Semantic Versioning 2.0
Tom Preston-Werner
<https://semver.org>
- [TMF630-1]
TMForum
REST API Design Guidelines
Part 1: Practical guidelines for RESTful APIs naming, CRUD, filtering, tasks, history, notifications
- [TMF630-2]
TMForum
REST API Design Guidelines
Part 2: Advanced guidelines for RESTful APIs polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- [TMF630-3]
TMForum
REST API Design Guidelines
Part 3: Guidelines for extending TMF Open API's with hypermedia support
- [TMF630-4]
TMForum
REST API Design Guidelines
Part 4: Advanced guidelines for import and export tasks and for entity versioning
- [TMF630-5]
TMForum

REST API Design Guidelines

Part 5: JSON patch extensions to manage arrays

□ [TMF630-6]

TMForum

REST API Design Guidelines

Part 6: JSON path extensions

□ [TMF630-7]

TMForum

REST API Design Guidelines

Part 7: JSON Schemas guideline

Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, sections may be included or omitted as desired; however, a Document History must always be included.

Document History

Release History

This section records the changes between this and the previous Official document release. The release number is the 'Marketing' number which this version of the document is first being assigned to.

Release Number	Date Modified	Modified by:	Description of changes
17.5.0	08-Nov-2017	Peter Norbury	first release of document
5.0.0	13-September-2025	Lutz Bettge	Upgraded to V5 of the Open APIs

Acknowledgments

Contributors to Prior Releases

Previous versions of this document were prepared by the following members of the TM Forum API team (in alphabetical order):

Andres Zoque	Netcracker	Contributor
John Morey	Ciena	Contributor
Jonathan Goldberg	Amdocs	Contributor
Maxime Delon	Orange	Contributor
Pierre Gauthier	TM Forum	Chief Architect, Team Leader
Veronique Mauneau	Orange	Contributor

Contributors to Current Release

The following members of the TM Forum API team contributed to this version of the document (in alphabetical order):

Florin Tene	Cityfibre	Contributor
Jonathan Goldberg	TMForum	Contributor
Lutz Bettge	Deutsche Telekom	Contributor, Editor
Olivier Arnaud	Orange	Contributor